285 HW 1 Report

Tomasz Frelek

Abstract

For this project I wrote a convolutional variational autoencoder in PyTorch, trained it on the Kaggle chest X-ray dataset at 64×64, and used it to generate new X-rays. The samples get the overall look of a chest X-ray right but come out blurry. They score an Inception Score of 1.434 (real X-rays get 1.965) and a 2048-d FID of 141.8. **Code is available at:** `https://github.com/tomaszfrelek2/285_hw1`

1 Methods

Model. The VAE has an encoder that maps an image x to a Gaussian over a latent code, $q(z \mid x) = \mathcal{N}(\mu(x), \operatorname{diag}\sigma^2(x))$, and a decoder that maps z back to an image. Training minimizes the usual negative ELBO, $\mathcal{L}(x) = -\mathbb{E}_{q(z|x)}[\log p(x \mid z)] + \beta \operatorname{KL}(q(z \mid x) \,\|\, \mathcal{N}(0, I))$, with the reparameterization trick ($z = \mu + \sigma \odot \epsilon$) so gradients flow through the sampling step. For the reconstruction term I used binary cross-entropy on pixel values in $[0, 1]$. Both terms are summed over the image, so all losses below are in nats per image. The encoder is four 4×4 stride-2 convolutions (32/64/128/256 channels, each with BatchNorm and LeakyReLU) followed by two linear layers for μ and $\log \sigma^2$. The decoder is the mirror image, built from transposed convolutions and ending in a sigmoid. To generate a new X-ray, I sample $z \sim \mathcal{N}(0, I)$ and decode it.

Data. I used the Kaggle chest X-ray dataset (Mooney, 2018), which has X-rays labeled NORMAL or PNEUMONIA. I ignored the labels and trained on all 5,216 `train` images (1,341 normal, 3,875 pneumonia). The official `val` split only has 16 images, so I used the 624-image `test` split for validation instead. Every image is converted to grayscale, resized and center-cropped to 64×64.

Table 1: Hyperparameter settings.

Hyperparameter	Value	Hyperparameter	Value
Resolution	64×64, grayscale	Optimizer	Adam, lr 10^{-3}, no weight decay
Latent dimension	128	LR schedule	cosine annealing to 0
Encoder channels	32 / 64 / 128 / 256	Batch size	128 (40 iterations/epoch)
Parameters	2.96M	Epochs	150
Reconstruction	BCE	KL weight β	1 (linear warm-up over 10 epochs)
Augmentation	none	Seed	0

Training and evaluation. Table 1 lists the hyperparameter values. Training was quick: 237 s total ($\sim$1.5 s per epoch) via a T4 provided on Google Colab. I kept the checkpoint with the lowest validation loss (epoch 139) and used it to generate the results below. To score the samples, I used the Inception Score and FID code from the PyTorch-Ignite tutorial (PyTorch-Ignite, n.d.). Images are upscaled to 299×299, the gray channel is copied to RGB, and an ImageNet-trained Inception-v3 network scores them. I generated 5,216 samples (one per training image), compared them against the real training images, and repeated this with 3 random seeds. I report two versions of FID: the tutorial's default, which compares Inception's 1000 class probabilities, and the standard version, which uses its 2048-d internal features and gives numbers on the familiar scale.

2 Results

Training. The loss curves are in Figure 1. In the very first epoch the KL term spikes (2.4×10^5 nats, cut off in the plot) because β starts near 0 during the KL warm-up, so nothing is holding the latent codes back yet. Once β ramps up, KL drops fast and settles around 18 nats, which suggests the model only really uses part of its 128 latent dimensions. At the best epoch, validation loss is 2601.8 nats (2583.7 reconstruction + 18.0 KL), compared with 2582.7 on the training set. That ∼19-nat gap (0.7%) is small, and validation loss levels off instead of going back up, so the model isn't really overfitting.

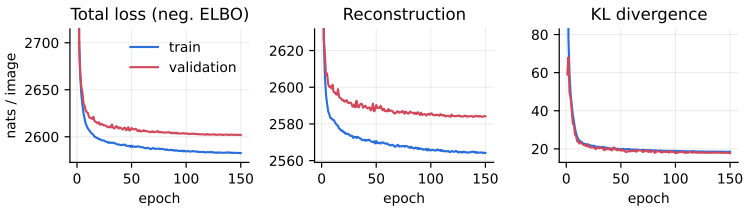


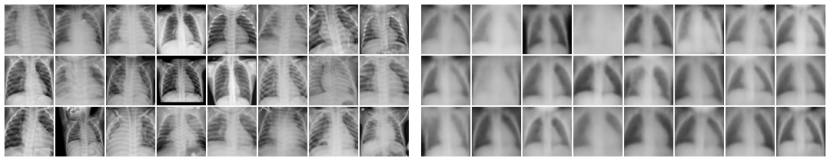
Figure 1: Training and validation loss per epoch (nats per image, computed with $\beta = 1$).

Table 2: IS and FID (mean ± std over 3 seeds, 5,216 images each). †Computed from 624 `test` vs. 624 `train` images.

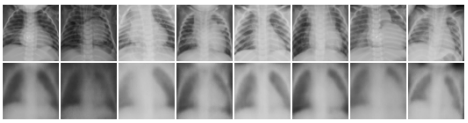
Images	IS ↑	FID (1000-d probs, tutorial) ↓	FID (2048-d pool) ↓
VAE samples	1.434 ± 0.002	0.0420 ± 0.0002	141.8 ± 0.4
Real X-rays (reference)	1.965	$0.060^{\dagger}$	$74.1^{\dagger}$

Scores. Table 2 has the IS and FID numbers, which barely change between seeds. IS is low even for real X-rays, because Inception was trained on everyday photos and lumps every X-ray into the same few ImageNet classes. So the useful comparison is against real images, and the samples score about 27% lower (1.43 vs. 1.97), which fits with them being blurrier. That same issue makes the tutorial's FID tiny (0.042), so it can't tell samples and real images apart very well. The standard 2048-d FID is more telling: 141.8 for the samples vs. 74.1 between two sets of real images.

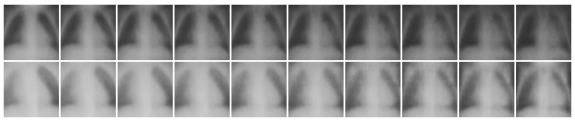
Generated images. The samples (Figure 2a) clearly look like chest X-rays: dark lungs, a bright heart and spine in the middle, collarbones and shoulders, and a believable range of brightness, contrast and positioning. But they're blurry and missing the ribs and fine texture, and the reconstructions (Figure 2b) have the same problem. This is a well-known VAE weakness: a pixel-by-pixel loss rewards predicting the average image, which comes out smooth. Interpolating between two encoded X-rays gives a smooth transition, so the latent space is nicely continuous. With more time, I'd try a perceptual or adversarial loss to sharpen things up, and condition on the normal/pneumonia label.



(a) Real training images (left) vs. VAE samples, $z \sim \mathcal{N}(0, I)$ (right).



(b) Validation images (top) and reconstructions.



(c) Latent interpolations between validation images.

Figure 2: Generated examples, reconstructions and latent interpolations (64×64).

References

P. Mooney. Chest X-ray images (pneumonia). Kaggle, 2018. https://www.kaggle.com/paultimothymooney/chest-xray-pneumonia.

PyTorch-Ignite. GAN evaluation with FID and IS. https://pytorch-ignite.ai/blog/gan-evaluation-with-fid-and-is/.